

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS:

Total Marks:50

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

*I **_P.Harika(192311243)_** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Harika

Annexure A

ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation**, including **lexical, syntax, semantic, logical, and runtime errors**. (BTL4) (10)

Given Program

```
#include <stdio.h>
```

```
int add(int a, int b) {  
    return a + b;  
}  
main() {  
    int lnum = 10;  
    float b = 5.5  
    int c;  
    c = add(10);  
    if(c = 20) {  
        printf("Value is 20\n")  
    }  
  
    d = c + 5;  
  
    int arr[5] = {1,2,3,4,5};  
    printf("%d\n", arr[5]);  
    int *p;  
    *p = 100;  
    int x = 10, y = 0;  
    int z = x / y;  
    while(x < 15) {  
        printf("%d\n", x);  
    }
```

```

        return 0;
    }

```

Tasks:

(a) Error Identification (Analysis Level)

1. Identify all the errors present in the program. For each error
 - int 1num = 10;
2. Specify the line number
 - 8
3. Clearly describe the nature of the error

Lexical Error (invalid identifier). Some books may also report it as a syntax error because the parser cannot parse the declaration, but it is primarily a **lexical error**.

(b) Classification of Errors (Understanding + Analysis)

1. Classify each identified error into the appropriate compiler phase:
2. Lexical Errors
- 3.

Error Reason	Line
--------------	------

int 1num = 10;	8 1num is an invalid identifier because identifiers cannot begin with a digit.
----------------	--

4. Syntax Errors
5. Semantic Errors
6. Logical Errors
- 7.
8. Runtime Errors

(c) Compilation Behavior (Evaluation Level)

1. Analyze which errors will be detected during compilation and which will only appear during execution.

2. Explain how early-stage errors (lexical/syntax) may prevent detection of later-stage errors.

(d) Code Correction (Application Level)

1. Rewrite the corrected version of the program such that:
2. All errors are removed
3. The program executes correctly
4. Proper syntax and semantics are maintained

(e) Impact Analysis (Evaluation Level)

1. Discuss the impact of at least three errors on program execution.
2. Explain how incorrect constructs like improper pointer usage, division by zero, and infinite loops affect system behavior.

(f) Advanced Compiler Perspective (Create Level)

1. Suggest how a compiler can be improved to:
2. Provide better error messages
3. Perform early detection of runtime errors
4. Propose at least two enhancements in compiler design for efficient Ensure:

ERROR ANALYSIS TASK –

Given Program – Error Identification and Analysis

The program contains **lexical, syntax, semantic, logical, and runtime errors**. These errors occur at different stages of compilation and execution.

(a) Error Identification (Analysis Level)

Line No.	Statement	Error Description
----------	-----------	-------------------

6	<code>main()</code>	Missing return type. It should be <code>int main()</code> .
7	<code>int 1num = 10;</code>	Invalid identifier. Variable names cannot begin with a digit.
8	<code>float b = 5.5</code>	Missing semicolon (;).
10	<code>c = add(10);</code>	Function <code>add()</code> requires two arguments, but only one is provided.
11	<code>if(c = 20)</code>	Assignment operator (=) used instead of comparison operator (==).
12	<code>printf("Value is 20\n")</code>	Missing semicolon (;).
14	<code>d = c + 5;</code>	Variable <code>d</code> is used without declaration.
16	<code>printf("%d\n", arr[5]);</code>	Array index out of bounds. Valid indices are <code>0–4</code> .
18	<code>*p = 100;</code>	Pointer <code>p</code> is uninitialized before dereferencing.
20	<code>int z = x / y;</code>	Division by zero (<code>y = 0</code>).
21–23	<code>while(x < 15)</code>	Infinite loop because <code>x</code> is never incremented.

Final Summary

The program contains errors at **multiple levels**. Lexical and syntax errors are generally detected first by the compiler. Semantic errors are identified during semantic analysis, while logical errors may remain even after successful compilation. Runtime errors such as **invalid pointer dereferencing, array out-of-bounds access, and division by zero** may only become visible during execution. Using better diagnostics, static analysis, CFG/data-flow analysis, and runtime safety checks can make compilers more reliable and user-friendly.

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lack clarity or proper structure	Poor presentation and difficult to understand